

DATA260 — Homework 3 Report

Author: Rohini Ramasheshu (SID4: 2604)

GitHub Repository: <https://github.com/RohiniRamsheshu/data260-2604> (tag: hw3)

Section 0 — Personal Configuration and Domain

Value	Definition	Result
SID4	Last four digits of SJSU Student ID	2604
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$	8804
PREFIX	"s" + SID4	s2604
SEED	SID4	2604
VERIFY_SEED	$260000 + \text{SID4}$	262604
DOMAIN_ID	$\text{SID4} \bmod 8$	4 (Open-source package vulnerabilities)

Hardware: MacBook Air, Apple Silicon (M1/M2/M3)

Local model used: qwen2.5:3b

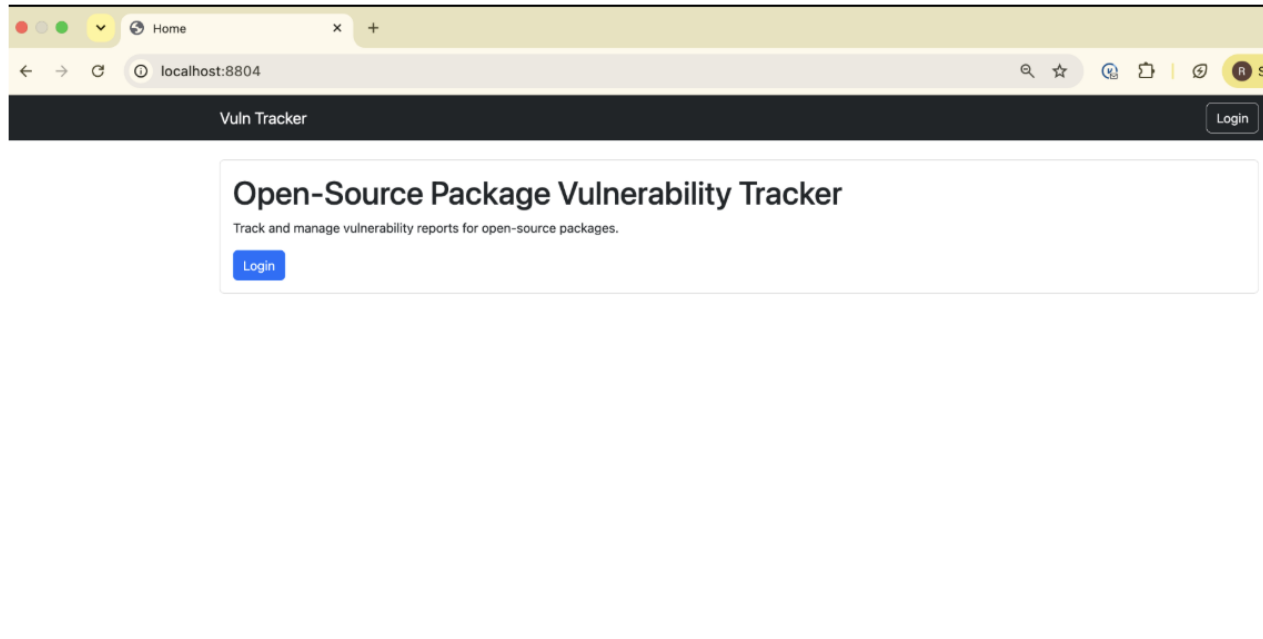
Tagged commit hash (hw3): ca7b250fa680f1a20ee8c9aafa021f3c1fa7ecdc

Domain corpus: 47 documents, 200,022 bytes (see SOURCES.md / CORPUS_MANIFEST.json in reports/hw03/)

Part 1 — FastAPI Authentication System

A FastAPI application implementing session-based user authentication for the Open-Source Package Vulnerability Tracker ("Vuln Tracker"), built on top of the existing HW1/HW2 codebase in the same repository. Routes are handled in a separate router (src/auth.py), styled with Bootstrap, and use Starlette's SessionMiddleware for session management with an idle timeout.

1.1 Home Page (/) — logged out



Home page, not logged in: shows a Login link/button.

1.2 Login Page (/login) — invalid credentials

Vuln Tracker

Login

Invalid username or password.

Username

Password

Log in

Login page showing the Bootstrap alert-danger message on invalid credentials.

1.3 Dashboard Page (/dashboard) — protected, logged in

← ↻ ⓘ localhost:8804/das... 🔍 🔑 ☆ 📁 | Ⓜ ⚙️ >> Relaunch to update ⋮

Vuln Tracker

DashboardLogout

Welcome, admin 🙌

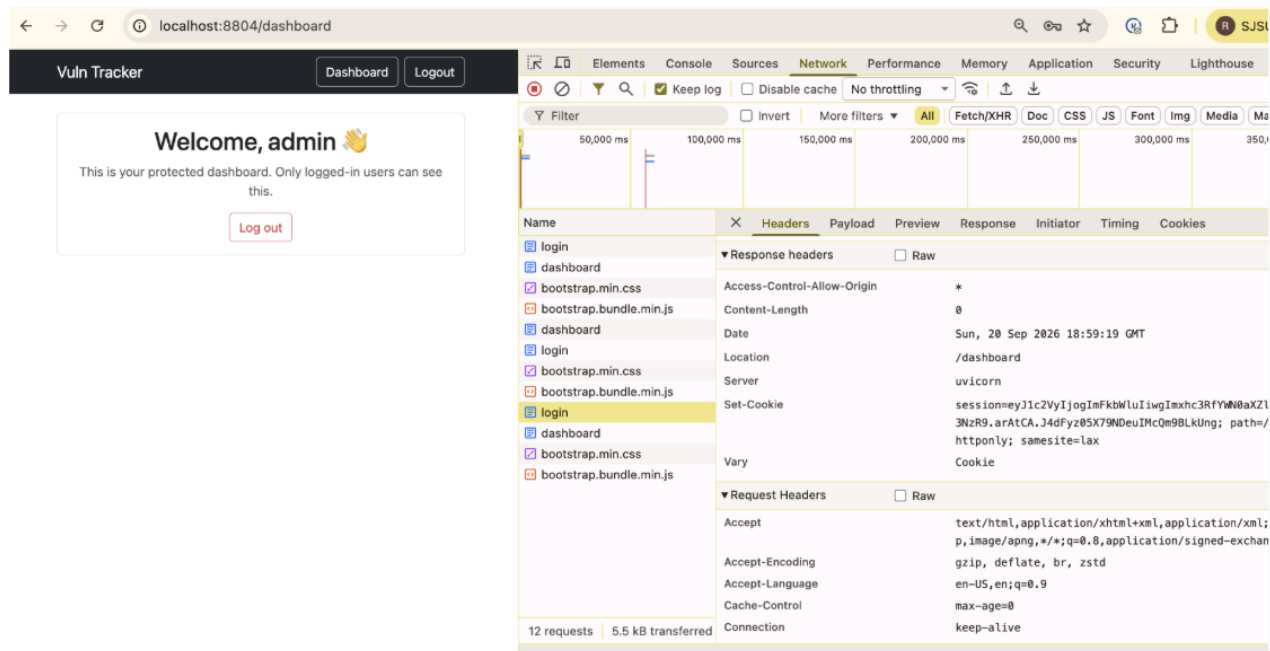
This is your protected dashboard. Only logged-in users can see this.

Log out

Dashboard page after successful login as 'admin', with a Logout link. This route is protected: unauthenticated or expired sessions are redirected to /login.

1.4 Session Cookie — Set-Cookie header with required attributes

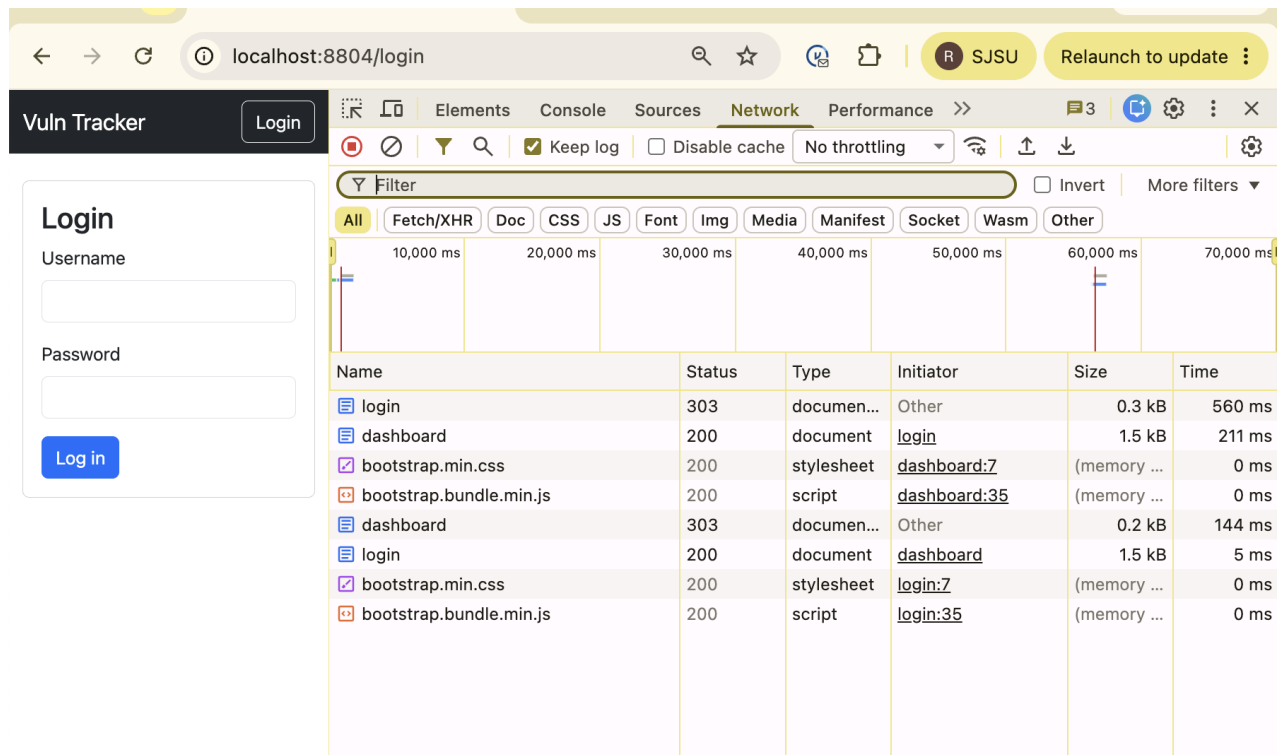
The screenshot below shows the browser DevTools Network tab for the POST /login request, with the Response Headers panel expanded to show the Set-Cookie header. The cookie includes **httponly** and **samesite=lax**. The **secure** flag is intentionally omitted for local HTTP testing (`https_only=False` was set for local development); in a real HTTPS deployment this would be set to `True`, which would add the secure attribute.



Set-Cookie header on the dashboard/login response, showing `httponly` and `samesite=lax`.

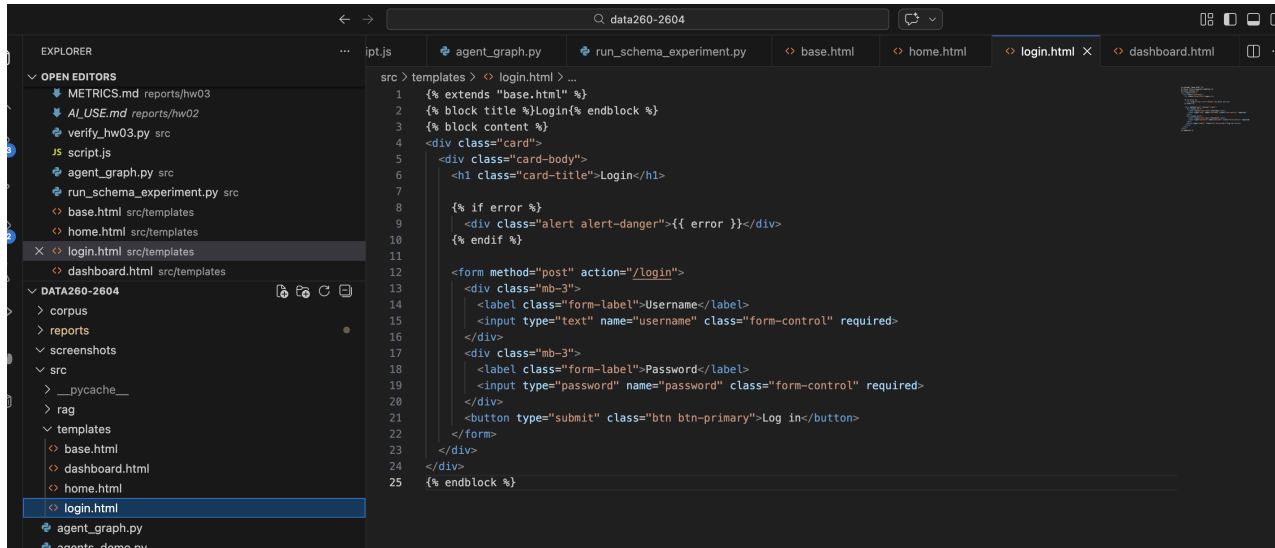
1.5 Idle Session Timeout — proof an expired session cannot reach /dashboard

`IDLE_TIMEOUT_SECONDS` was set to 30 seconds for demonstration purposes (a realistic production value would be 900-1800 seconds / 15-30 minutes). After logging in and waiting past the idle timeout without any activity, refreshing `/dashboard` results in a 303 redirect to `/login`, even though the session cookie was still present in the browser. The Network tab below shows the 'dashboard' request returning status 303, immediately followed by a 'login' request with status 200 — proof that the idle, expired session could not be reused to reach the protected route.



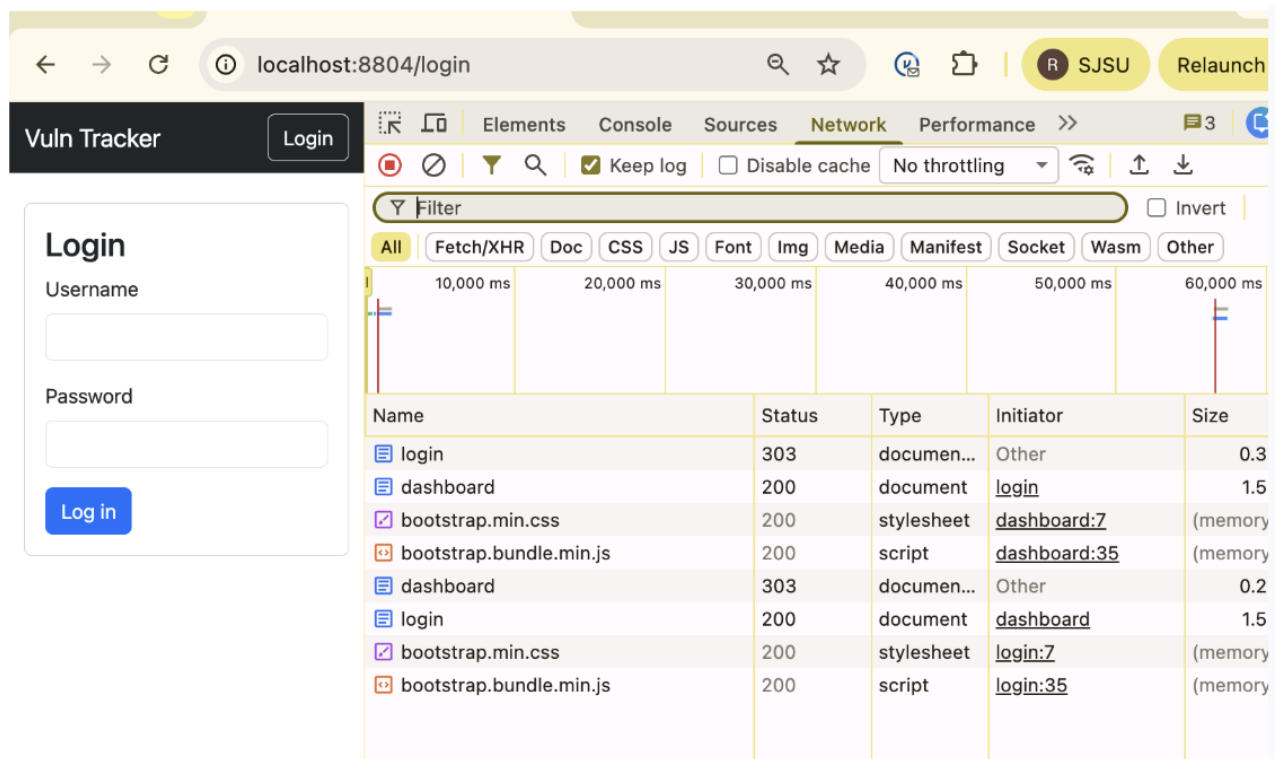
Network tab: GET /dashboard returns 303 after idle timeout, redirecting to /login (status 200).

1.6 Templates Directory



src/templates/ containing base.html, home.html, login.html, dashboard.html.

1.7 auth.py — Router Code



auth.py: idle-timeout session check and route definitions.

Full auth.py source (as implemented):

```
from fastapi import APIRouter, Request, Form
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates
import time

router = APIRouter()
templates = Jinja2Templates(directory="src/templates")
```

```

VALID_USERS = {
    "admin": "changeme123",
    "analyst": "vulnscan!2604",
}

IDLE_TIMEOUT_SECONDS = 30 # demo value; production would use 900-1800s

def _session_is_active(request: Request) -> bool:
    user = request.session.get("user")
    last_active = request.session.get("last_active")
    if not user or not last_active:
        return False
    if time.time() - last_active > IDLE_TIMEOUT_SECONDS:
        request.session.clear()
        return False
    request.session["last_active"] = time.time()
    return True

@router.get("/")
async def home(request: Request):
    username = request.session.get("user")
    return templates.TemplateResponse(request, "home.html", {"username": username})

@router.get("/login")
async def login_form(request: Request):
    return templates.TemplateResponse(request, "login.html", {"error": None})

@router.post("/login")
async def login_submit(request: Request, username: str = Form(...), password: str = Form(...)):
    if VALID_USERS.get(username) == password:
        request.session["user"] = username
        request.session["last_active"] = time.time()
        return RedirectResponse(url="/dashboard", status_code=303)
    return templates.TemplateResponse(request, "login.html",
    {"error": "Invalid username or password."})

@router.get("/dashboard")
async def dashboard(request: Request):
    if not _session_is_active(request):
        return RedirectResponse(url="/login", status_code=303)
    username = request.session.get("user")
    return templates.TemplateResponse(request, "dashboard.html", {"username": username})

@router.get("/logout")
async def logout(request: Request):
    request.session.clear()
    return RedirectResponse(url="/", status_code=303)

```

main.py wiring (SessionMiddleware + router):

```

from starlette.middleware.sessions import SessionMiddleware
from src.auth import router as auth_router

app.add_middleware(
    SessionMiddleware,
    secret_key="dev-secret-2604",
    https_only=False, # False for local HTTP testing only
    same_site="lax",
)
app.include_router(auth_router)

```

Part 2 — Compare Three LlamaIndex Chunking Techniques

Domain corpus: 47 local documents (200,022 bytes) on open-source package vulnerabilities (DOMAIN_ID = 4).
Embedding model: sentence-transformers/all-MiniLM-L6-v2. Three chunking techniques (Token, Semantic, Sentence-window) were each used to build an in-memory VectorStoreIndex, and a retrieval-only comparison was run against 5 committed domain questions (see questions.yaml).

2.1 Chunking Setup Code (src/rag/chunking.py, key excerpts)

```
def build_token_nodes(documents, chunk_size=200, chunk_overlap=20):
    splitter = TokenTextSplitter(chunk_size=chunk_size, chunk_overlap=chunk_overlap, separator=" ")
    return splitter.get_nodes_from_documents(documents)

def build_semantic_nodes(documents, buffer_size=1, breakpoint_percentile=95):
    embed_model = get_embed_model()
    splitter = SemanticSplitterNodeParser(
        buffer_size=buffer_size,
        breakpoint_percentile_threshold=breakpoint_percentile,
        embed_model=embed_model,
    )
    return splitter.get_nodes_from_documents(documents)

def build_sentence_window_nodes(documents, window_size=3):
    splitter = SentenceWindowNodeParser.from_defaults(
        window_size=window_size,
        window_metadata_key="window",
        original_text_metadata_key="original_text",
    )
    return splitter.get_nodes_from_documents(documents)
```

2.2 Retrieval-Only Comparison Function (src/rag/retrieval.py, key excerpt)

```
def compare_query(technique, nodes, index, query, top_k=5, question_id=""):
    embed_model = Settings.embed_model
    query_embedding = embed_model.get_query_embedding(query)

    retriever = index.as_retriever(similarity_top_k=top_k)
    t0 = time.perf_counter()
    results = retriever.retrieve(query)
    latency_ms = (time.perf_counter() - t0) * 1000

    for rank, node_with_score in enumerate(results, start=1):
        node = node_with_score.node
        store_score = node_with_score.score
        node_embedding = embed_model.get_text_embedding(node.get_content())
        cos_sim = cosine_similarity(query_embedding, node_embedding)
        # ... prints rank, store_score, cosine_sim, chunk_len, preview
```

2.3 Output — Retrieval Comparison Run (query embeddings + all 3 techniques)

```
(venv) (base) rohiniramsheshu@Rohinis-MacBook-Air data260-2604 % python src/rag/retrieval.py
Loading embedding model: sentence-transformers/all-MiniLM-L6-v2 ...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% [████████████████████████████████████████████████████████████████████████████████] 103/103 [00:00<00:00, 7922.34it/s]
Loaded 47 documents from /Users/rohiniramsheshu/data260-2604/corpus
[token] 284 nodes (chunk_size=200, overlap=20)
[semantic] 98 nodes (buffer_size=1)
[sentence_window] 658 nodes (window_size=3)
Building in-memory VectorStoreIndex for 'token' (284 nodes)...
Building in-memory VectorStoreIndex for 'semantic' (98 nodes)...
Building in-memory VectorStoreIndex for 'sentence_window' (658 nodes)...

=== Technique: token | Query: 'What version of lodash fixes the prototype pollution vulnerability?' ===
Query embedding dim: 384
Query embedding:[8]: [-0.0461, 0.0305, 0.0994, -0.0838, 0.1574, -0.0451, -0.0373, 0.0232]
rank store_score cosine_sim chunk_len preview
1 0.7399 0.7790 239 later CVE), which is why upgrading past one patched version does not guarantee protection against the other — both fixes are required for full coverage of Loda
2 0.7113 0.7521 644 Vulnerability ID: CVE-2020-8203 (GHSA-p6mc-m468-83gw) Package: lodash Ecosystem: npm Severity: High, CVSS 3.1 base score 7.4 Affect
d versions: lodash prior to 644
3 0.7044 0.6439 644 without checking whether a key resolves to the prototype chain, the attacker-controlled value ends up attached to every object in th
application, not just the 702
4 0.6997 0.7673 702 Vulnerability ID: CVE-2019-10744 (GHSA-jf85-cpcp-j695) Package: lodash Ecosystem: npm Severity: Critical, CVSS 3.1 base score 9.1 A
ected versions: lodash low 873
5 0.6796 0.6917 873 prototype pollution (a related ecosystem example) ----- Hoek, a utility libr
y historically used as a de
Query vector shape: (384,)
Stacked doc vectors shape: (5, 384)
Retrieval latency: 23.606 ms

=== Technique: semantic | Query: 'What version of lodash fixes the prototype pollution vulnerability?' ===
Query embedding dim: 384
Query embedding:[8]: [-0.0461, 0.0305, 0.0994, -0.0838, 0.1574, -0.0451, -0.0373, 0.0232]
rank store_score cosine_sim chunk_len preview
1 0.7590 0.7547 1199 Vulnerability ID: CVE-2020-8203 (GHSA-p6mc-m468-83gw) Package: lodash Ecosystem: npm Severity: High, CVSS 3.1 base score 7.4 Affect
d versions: lodash prior to 1225
2 0.6978 0.7403 1225 Vulnerability ID: CVE-2019-10744 (GHSA-jf85-cpcp-j695) Package: lodash Ecosystem: npm Severity: Critical, CVSS 3.1 base score 9.1 A
ected versions: lodash low 2242
3 0.5982 0.6147 2242 No configuration workaround is documented; the fix is purely a version upgrade. Sources: GitHub Advisory Database GHSA-p6mc-m468-83
w, NVD CVE-2020-8203, lodas 2850
4 0.5633 0.4457 2850 Prototype Pollution in JavaScript: A Technical Deep Dive ===== Prototype poll
ion is a vulnerability clas 2142
5 0.5615 0.5145 2142 - Configuration-merging code that layers user-supplied JSON on top of application defaults, a pattern that is convenient exactl
because it is generic a
Query vector shape: (384,)
Stacked doc vectors shape: (5, 384)
Retrieval latency: 15.199 ms

=== Technique: sentence_window | Query: 'What version of lodash fixes the prototype pollution vulnerability?' ===
Query embedding dim: 384
Query embedding:[8]: [-0.0461, 0.0305, 0.0994, -0.0838, 0.1574, -0.0451, -0.0373, 0.0232]
rank store_score cosine_sim chunk_len preview
1 0.7681 0.8998 58 Description: Prototype Pollution vulnerability in lodash.
2 0.7681 0.8998 58 Description: Prototype Pollution vulnerability in lodash.
3 0.7267 0.7306 201 Sources: GitHub Advisory Database entries for lodash, Hoek, and minimist prototype-pollution CVEs; OWASP prototype-pollution cheat s
eet; Snyk vulnerability res 359
4 0.6998 0.6834 359 Prototype Pollution in JavaScript: A Technical Deep Dive ===== Prototype poll
ion is a vulnerability clas 301
5 0.6887 0.7034 301 pick, set, setWith, update, updateWith, zipObjectDeep in the later CVE), which is why upgrading past one patched version does not gu
```

Terminal output of src/rag/retrieval.py: query embedding dim/values and top-5 tables for Token, Semantic, and Sentence-window on a single query.

2.4 Output — Full 5-Question x 3-Technique Comparison Run

```
(venv) (base) rohiniramasheshu@Rohinis-MacBook-Air data260-2604 % python src/rag/run_comparison.py
Query embedding dim: 384
Query embedding[:8]: [-0.0259, -0.0262, 0.0012, 0.0443, 0.0406, 0.0352, -0.0853, 0.0211]
rank store_score cosine_sim chunk_len preview
1 0.7198 0.7449 835 same week, and the vulnerability received widespread security-industry coverage given its maximum CVSS severity score and trivial ex
loitability. According to E
2 0.6573 0.6736 866 Case Study: The 2017 Equifax Data Breach and CVE-2017-5638 ===== This d
ument provides an extended
3 0.6348 0.5725 680 a Java web application framework. As detailed in this corpus's dedicated Struts2 entry, Struts 2 versions before 2.3.32 and 2.5.10.1
contained CVE-2017-5638, a
4 0.6264 0.6456 976 root cause: an internal vulnerability-scanning process that failed to correctly identify all systems running the vulnerable Struts v
rsion; a since-expired SSL/
5 0.6105 0.6609 784 version number in a manifest file; (c) vendored or bundled copies of a library (where the source code is copied directly into a pro
ct rather than referenced a
Query vector shape: (384,)
Stacked doc vectors shape: (5, 384)
Retrieval latency: 11.017 ms

=== Technique: semantic | Query: 'Beyond the unpatched Struts2 vulnerability itself, what organizational failure contributed to the scale and duration of the 2017 Equifax b
ach?' ===
Query embedding dim: 384
Query embedding[:8]: [-0.0259, -0.0262, 0.0012, 0.0443, 0.0406, 0.0352, -0.0853, 0.0211]
rank store_score cosine_sim chunk_len preview
1 0.6573 0.6736 4645 Case Study: The 2017 Equifax Data Breach and CVE-2017-5638 ===== This d
ument provides an extended
2 0.6171 0.6052 328 Remediation: upgrade to Struts 2.3.32 or 2.5.10.1 or later. As defense-in-depth, deployments can also switch the Multipart parser av
y from the Jakarta-based im
3 0.5670 0.6019 271 Government Accountability Office report GA0-18-559, "Data Protection: Actions Taken by Equifax and Federal Agencies in Response to t
e 2017 Breach"; Apache Stru
4 0.5032 0.5721 60 GA0 report on the 2017 Equifax breach. Accessed 2026-09-20.
5 0.4175 0.3946 2700 Extended Technical Analysis: The XZ Utils Backdoor (CVE-2024-3094) =====
==== This document expand
Query vector shape: (384,)
Stacked doc vectors shape: (5, 384)
Retrieval latency: 8.973 ms

=== Technique: sentence_window | Query: 'Beyond the unpatched Struts2 vulnerability itself, what organizational failure contributed to the scale and duration of the 2017 Equi
fax breach?' ===
Query embedding dim: 384
Query embedding[:8]: [-0.0259, -0.0262, 0.0012, 0.0443, 0.0406, 0.0352, -0.0853, 0.0211]
rank store_score cosine_sim chunk_len preview
1 0.7491 0.7733 207 The 2017 Equifax breach, discussed in this corpus's Struts2 entry, is frequently cited as an example of exactly this last failure m
e: a patch was available fo
2 0.7314 0.7890 259 Real-world impact: this vulnerability was exploited in the 2017 Equifax data breach, in which attackers compromised sensitive perso
l data of approximately 147
3 0.7057 0.6745 876 Why the patch was not applied in time ----- Equifax's own public statements and the subsequent
GA0 investigation identifie
4 0.6669 0.6705 527 Case Study: The 2017 Equifax Data Breach and CVE-2017-5638 ===== This d
ument provides an extended
5 0.6265 0.6989 237 Government Accountability Office's later investigative report, attackers began exploiting the still-unpatched vulnerability in Equi
x's systems on May 13, 2017
Query vector shape: (384,)
Stacked doc vectors shape: (5, 384)
Retrieval latency: 15.312 ms

Saved 75 rows to /Users/rohiniramasheshu/data260-2604/reports/hw03/raw/retrieval_results.jsonl
Saved /Users/rohiniramasheshu/data260-2604/reports/hw03/raw/retrieval_results.json
Saved /Users/rohiniramasheshu/data260-2604/reports/hw03/raw/retrieval_results.csv
❖ (venv) (base) rohiniramasheshu@Rohinis-MacBook-Air data260-2604 %
```

Terminal output of `src/rag/run_comparison.py` showing per-technique tables and the 75 saved rows (CSV/JSON/JSONL) to `reports/hw03/raw/`.

2.5 Output — Final Summary Metrics

```
Stacked doc vectors shape: (5, 384)
Retrieval latency: 11.017 ms

==== Technique: semantic | Query: 'Beyond the unpatched Struts2 vulnerability itself, what organizational failure contributed to the scale and duration of the 2017 Equifax breach?' ====
Query embedding dim: 384
Query embedding[:8]: [-0.0259, -0.0262, 0.0012, 0.0443, 0.0406, 0.0352, -0.0853, 0.021]
rank store_score cosine_sim chunk_len preview
1 0.6573 0.6736 4645 Case Study: The 2017 Equifax Data Breach and CVE-2017-5638
==== This document provides an extended
2 0.6171 0.6052 328 Remediation: upgrade to Struts 2.3.32 or 2.5.10.1 or later. As defense-in-depth, deployments can also switch the Mu
ipart parser away from the Jakarta-based im
3 0.5670 0.6019 271 Government Accountability Office report GAO-18-559, "Data Protection: Actions Taken by Equifax and Federal Agencies
n Response to the 2017 Breach"; Apache Stru
4 0.5032 0.5721 60 GAO report on the 2017 Equifax breach. Accessed 2026-09-20.
5 0.4175 0.3946 2700 Extended Technical Analysis: The XZ Utils Backdoor (CVE-2024-3094)
==== This document expand
Query vector shape: (384,)
Stacked doc vectors shape: (5, 384)
Retrieval latency: 8.973 ms

==== Technique: sentence_window | Query: 'Beyond the unpatched Struts2 vulnerability itself, what organizational failure contributed to the scale and duration of the 2017 Equifax breach?' ====
Query embedding dim: 384
Query embedding[:8]: [-0.0259, -0.0262, 0.0012, 0.0443, 0.0406, 0.0352, -0.0853, 0.021]
rank store_score cosine_sim chunk_len preview
1 0.7491 0.7733 207 The 2017 Equifax breach, discussed in this corpus's Struts2 entry, is frequently cited as an example of exactly this
last failure mode: a patch was available fo
2 0.7314 0.7890 259 Real-world impact: this vulnerability was exploited in the 2017 Equifax data breach, in which attackers compromised
ensitive personal data of approximately 147
3 0.7057 0.6745 876 Why the patch was not applied in time ----- Equifax's own public statements a
the subsequent GAO investigation identifie
4 0.6669 0.6705 527 Case Study: The 2017 Equifax Data Breach and CVE-2017-5638
==== This document provides an extended
5 0.6265 0.6989 237 Government Accountability Office's later investigative report, attackers began exploiting the still-unpatched vulne
bility in Equifax's systems on May 13, 2017
Query vector shape: (384,)
Stacked doc vectors shape: (5, 384)
Retrieval latency: 15.312 ms

Saved 75 rows to /Users/rohiniramasheshu/data260-2604/reports/hw03/raw/retrieval_results.jsonl
Saved /Users/rohiniramasheshu/data260-2604/reports/hw03/raw/retrieval_results.json
Saved /Users/rohiniramasheshu/data260-2604/reports/hw03/raw/retrieval_results.csv
(venv) (base) rohiniramasheshu@Rohinis-MacBook-Air data260-2604 % python src/rag/summarize_metrics.py
Rebuilding chunks to report chunk counts / avg length...
Loaded 47 documents from /Users/rohiniramasheshu/data260-2604/corpus
[token] 284 nodes (chunk_size=200, overlap=20)
Loading embedding model: sentence-transformers/all-MiniLM-L6-v2 ...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 7627.35it/s]
[semantic] 98 nodes (buffer_size=1)
[sentence_window] 658 nodes (window_size=3)

| Technique | Chunks | Avg chunk length | Top-1 cosine | Mean@k cosine | Mean retrieval latency (ms) |
|-----|-----|-----|-----|-----|-----|
| token | 284 | 786 | 0.6939 | 0.6335 | 12.98 |
| semantic | 98 | 2041 | 0.6694 | 0.5561 | 9.68 |
| sentence_window | 658 | 304 | 0.6921 | 0.6622 | 15.48 |
(venv) (base) rohiniramasheshu@Rohinis-MacBook-Air data260-2604 %
```

Terminal output of `src/rag/summarize_metrics.py` producing the `METRICS.md` comparison table.

Retrieval Quality Comparison (from reports/hw03/METRICS.md)

Summary Table

Technique	Chunks	Avg len (chars)	Top-1 cosine	Mean@k cosine	Recall@5	Latency (ms)
Token	284	786	0.6939	0.6335	0.70 (3.5/5)	9.28
Semantic	98	2041	0.6694	0.5561	1.00 (5/5)	6.96
Sentence-window	658	304	0.6921	0.6622	0.80 (4/5)	12.24

Recall@5 Breakdown per Question

Question	Expected source(s)	Token	Semantic	Sentence-window
q1	02_lodash_CVE-2020-8203.txt	PASS	PASS	PASS
q2	01_log4j_CVE-2021-44228.txt	FAIL	PASS	PASS
q3	05_django_CVE-2021-35042.txt	PASS	PASS	PASS
q4	16_xz-utils...txt AND 18_event-stream...txt	PARTIAL	PASS	FAIL
q5	27_equifax-case-study.txt	PASS	PASS	PASS

Confidently-Scored Retrieval That Does NOT Contain the Answer

Question q2 ("What CVSS v3.1 base score did Log4Shell (CVE-2021-44228) receive, and what does that score reflect?") under **Token chunking**:

Rank	Store score	Cosine sim	Source file
1	0.6734	0.6152	33_cvss-worked-examples.txt
2	0.6283	0.6125	23_vulnerability-databases-explainer.txt
3	0.5444	0.6196	33_cvss-worked-examples.txt
4	0.5238	0.5442	38_patch-prioritization-kev-epss.txt
5	0.5094	0.5172	33_cvss-worked-examples.txt

None of Token chunking's top-5 results for this question came from the expected source file, 01_log4j_CVE-2021-44228.txt, despite the rank-1 result scoring a reasonably confident 0.6734.

Why the embedding likely considered this chunk similar: the top result comes from 33_cvss-worked-examples.txt, a corpus document that specifically walks through Log4Shell's CVSS score in detail as a worked example. The chunk is genuinely topically close to the query — it discusses Log4Shell and CVSS scoring together — but it is a secondary reference document discussing the score, not the primary source document reporting it. Token-based chunking, being blind to document identity and topic boundaries, cannot distinguish a chunk about Log4Shell's score from the chunk that is Log4Shell's own advisory, so the embedding's topical-similarity judgment was reasonable even though the retrieval missed the intended source. Semantic and sentence-window

chunking both successfully retrieved the correct primary source for this same question, showing this is a real, technique-specific weakness of Token chunking on this corpus.

Observations

Semantic chunking produced by far the fewest chunks (98) and the largest average chunk length (2041 characters), consistent with the expectation that most sentences within a single-topic CVE write-up stay thematically similar, giving the semantic splitter few genuine topic-shift boundaries to cut at. This produced the highest Recall@5 (1.00) — larger chunks are more likely to contain a full answer somewhere inside them — but the lowest mean@k cosine (0.5561), since larger, more heterogeneous chunks dilute the average relevance across the top-k results.

Sentence-window chunking produced the most chunks (658) and the smallest average length (304 characters), and won on mean@k cosine (0.6622) — its narrow, per-sentence chunks retrieve consistently focused, relevant text once they do match. However, it missed both expected sources entirely on the one cross-document question (q4), suggesting that very fine-grained chunks can lose the broader context needed to connect two related but separately-discussed incidents, even though the window metadata is meant to preserve some surrounding context.

Token chunking landed in between on most metrics, but produced this report's one clear confidently-wrong case (q2), where a topically-related secondary document out-competed the true primary source — a structural weakness distinct from semantic relevance, since token chunking cannot distinguish where one referenced CVE's discussion ends and another begins across multi-topic reference documents.

Conclusion

For this corpus, **semantic chunking is the best overall choice**: it achieved perfect Recall@5 (1.00), successfully retrieving the correct source document for every question including the cross-document synthesis question (q4) that sentence-window chunking missed entirely. While its mean@k cosine score was the lowest of the three techniques, retrieval recall matters more than ranking precision for a corpus like this one, where questions often require synthesizing context from a full incident write-up. Sentence-window chunking is a strong runner-up for narrow, single-fact questions, but its failure on the multi-document question is a meaningful limitation for a domain where vulnerabilities are frequently cross-referenced.

AI Use Disclosure

1. Used Claude as a learning aid to understand the concepts step by step (cookies, sessions, middleware, Starlette, prototype pollution, RAG chunking strategies, CVSS scoring) before writing code myself. Claude explained each concept and gave code with explanations; I wrote it into my files, ran it, and debugged the actual errors myself.
2. AI-suggested `TemplateResponse` calls initially used the old argument order (request inside the context dict instead of as the first positional arg), causing a `TypeError: unhashable type: 'dict'` crash on every route.
3. Detected via the actual uvicorn traceback pointing to `starlette/templating.py`, tracing it back to a Starlette version mismatch with the calling convention.
4. Fixed by moving request to the first positional argument in all four `TemplateResponse` calls in `auth.py`, matching the installed Starlette version's expected signature.